



Auto Controller 2 Manual

by brianuuuSonic

Getting Started.....	2
1. What is Auto Controller 2?.....	2
2. Hardware Requirement.....	2
2.1 Micro-Controllers.....	2
2.2 Optional Components.....	3
2.3 Capture Cards.....	4
2.4 HDMI Cables.....	5
2.5 CP210x Serial Converter Chip.....	5
2.6 Male-to-Female Jumper Wires.....	5
2.7 Mini-Grabbers.....	6
3. Hardware Setup.....	6
3.1 Installing .hex into Micro-Controller.....	6
3.2 Connecting Everything Together.....	9
4. Using Auto Controller 2.....	12
4.1 Disconnecting Other Controllers.....	12
4.2 Basic Connection.....	13
4.2 Testing Keyboard Controls.....	14
4.3 Global Settings.....	15
4.4 Nintendo Switch System Settings.....	15
4.5 Running a Program.....	15
4.6 Testing Camera Delay.....	15
5. Discord Connection.....	16
5.1 Use Cases.....	16
5.2 Discord Settings.....	17

Getting Started

1. What is Auto Controller 2?

Auto Controller 2 is a collection of programs that uses a micro-controller emulated as a Nintendo Switch Pro Controller and automate various tasks on games on the Nintendo Switch. These programs make use of visual and audio feedback from capture card to perform complicated tasks.

2. Hardware Requirement

Note that Auto Controller 2 is only available on Windows, there is no support for MacOS and Linux.

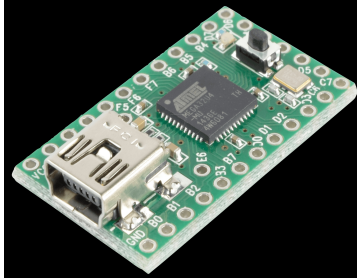
2.1 Micro-Controllers

You will need to buy one of the following micro-controllers:

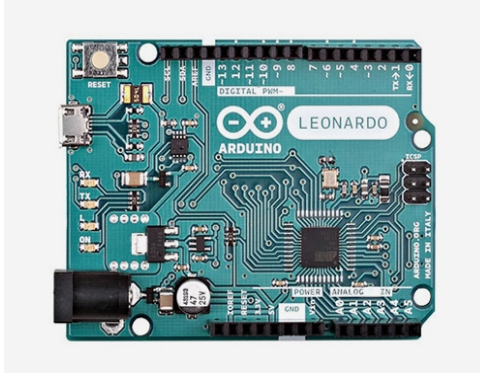
1. **Arduino Uno R3** (highly recommended, tested)



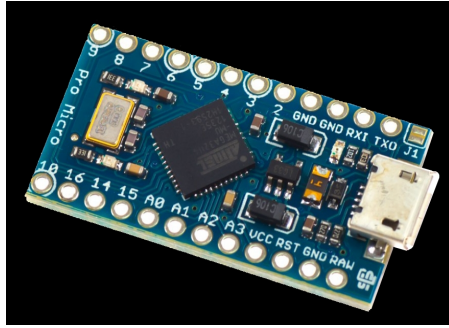
2. **Teensy 2.0/Teensy++ 2.0** (recommend, more expensive)



3. **Arduino Leonardo** (preferred over Pro Micro)



4. Arduino Pro Micro



There are knockoffs for most Arduino boards as a cheaper alternative like Elegoo etc., **but sometimes the microchip required (atmega16u2) may not be probably bootloaded by the manufacturer so you are HIGHLY RECOMMENDED to buy the Arduino from the official site instead.**

2.2 Optional Components

This section lists components you may/may not need depending on the micro-controller you have bought.

1. **Metal Wires/Paper Clips (Arduino Uno R3/Pro Micro):** This is needed in order to restart and install firmware into the board
2. **USB-A to USB-B Connector (Arduino Uno R3 Only):** Some sellers may not provide the connection wire for Arduino to PC, for Uno R3, you may need to buy this separately



3. **USB 2.0 to USB Micro (Arduino Pro Micro/Leonardo):** For these two boards, you may need to buy this wire, if you are an Android user, this is the regular charging cable (before USB-C migration)
4. **USB 2.0 to USB Mini (Teensy):** For teensy you may need to buy this wire, this is NOT the same cable that charges phones, check the image below



2.3 Capture Cards

There are three types of capture cards you can buy, but the general rule of thumb is it should be using **USB 3.0** if possible, to get the best video quality and to minimize latency, the minimum resolution needs to be **720p 60fps**.

- **Pass-Through:** This is the best option for best video quality and if you are intended to do livestreams, but it can also be the most expensive ranging from \$40 to \$150. This can output video to PC and TV at the same time with very low latency, example brands like Elgato and Avermedia (right is the one I use)



- **Loop-Out:** This is a cheaper (around \$30) middle ground option that also provides output for PC and TV, however the output to PC can have larger enough latency that may cause direct control from PC very difficult, you should at least get one with USB 3.0 if you are going for this option.



- **No TV Output:** This is the cheapest option (around \$20) but have no TV output, meaning you can only play using the video display on your computer which will largely depend on the latency of the output. The video quality can also vary and may cause detection issues in Auto Controller 2. If you are able to find those with **USB 3.0** it should be sufficient enough to be used.



Once you got the capture card you should follow the instructions provided by the manufacturer and make sure everything is working before continuing. You can also check and see if you are able to see video output with streaming softwares like OBS or VLC Player.

2.4 HDMI Cables

You should already have one when you first buy the Switch, but you may need a 2nd one for both capture card input and output.

2.5 CP210x Serial Converter Chip

This is the **most important component** for using Auto Controller 2, this is responsible to communicate between your Arduino/Teensy and computer, this should cost you only a few dollars. This chip belongs to CP210x family so CP2102 should also work but CP2104 is the one tested. There are also alternatives for UART modules (with TXD, RXD and GND pins) but unless you are sure it will work I just recommend you to get CP2104 to be safe.

After getting it, plugging it to your computer should automatically install the driver, if not you can get them here: <https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers> Look for “CP210x Windows Drivers”.

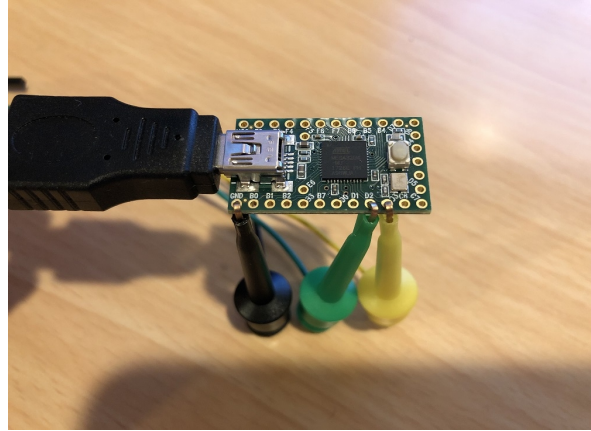
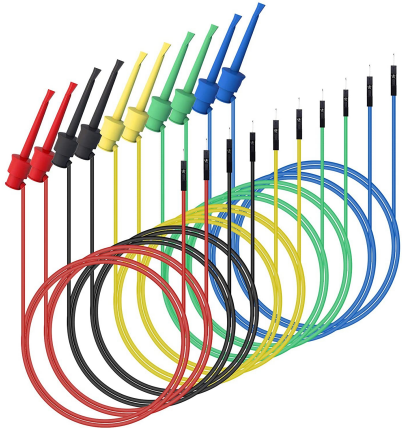


2.6 Male-to-Female Jumper Wires

When you buy CP210x usually it comes with female-to-female wires, but if you have **Arduino Uno R3/Leonardo** which also has female headers, you will need male-to-female jumper wires to connect between them.

2.7 Mini-Grabbers

You will most likely need this if you have Teensy or Arduino Pro Micro. These are jumper wires that has a hook on one end so you can hook on to a pinhole without headers, which Teensy does not provide unlike Uno R3. Alternatively you can solder headers onto the board but unless you know how to do soldering, do not bother.



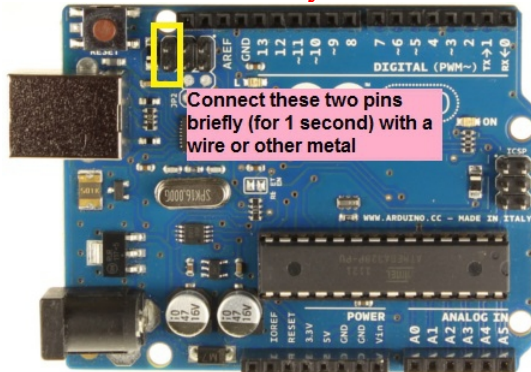
3. Hardware Setup

3.1 Installing .hex into Micro-Controller

This section changes depending on what microcontroller you own, please skip to the appropriate section.

Arduino Uno R3:

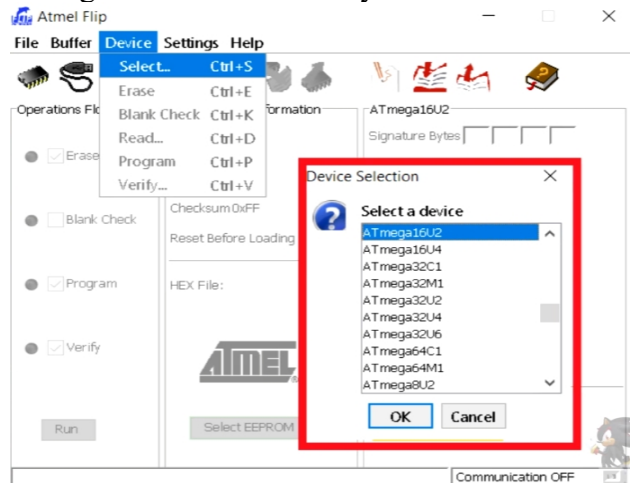
1. **(One time only)** Download and install FLIP from here:
<https://www.microchip.com/en-us/development-tool/flip>
You may also need to install Java Runtime Environment as suggested
2. Connect Uno R3 with USB-A to USB-B cable to your computer, use a conducting object to briefly connect the RESET and GND pin, you should hear the USB unplug and plugin sound, this process is call “**Flashing**” or “**DFU Mode**”. If you don’t see those 2 pins, you might have bought the wrong board.
You MUST do this every time .hex file is updated for Auto Controller 2



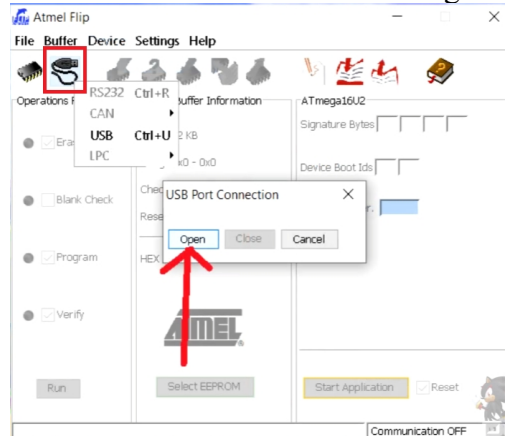
3. **(One time only)** The first time you use this, you must install the driver for Atmega16u2 chip before you can use it. Go to **Device Manager** in Control Panel, under Other devices you should see an “Unknown Device”, **if you don’t see it**

despite flashing the two pins, your board is defective and cannot be used, you are likely using a . Right-click -> Update Device Software -> Browse my computer for driver software, select the folder *C:/Program Files (x86)/Atmel/Flip 3.4.7/usb* and install, the device should now be registered as **Atmega16u2**

4. Run FLIP program, go to Device -> Select... And select ATmega16U2, this setting should remain after you did it once

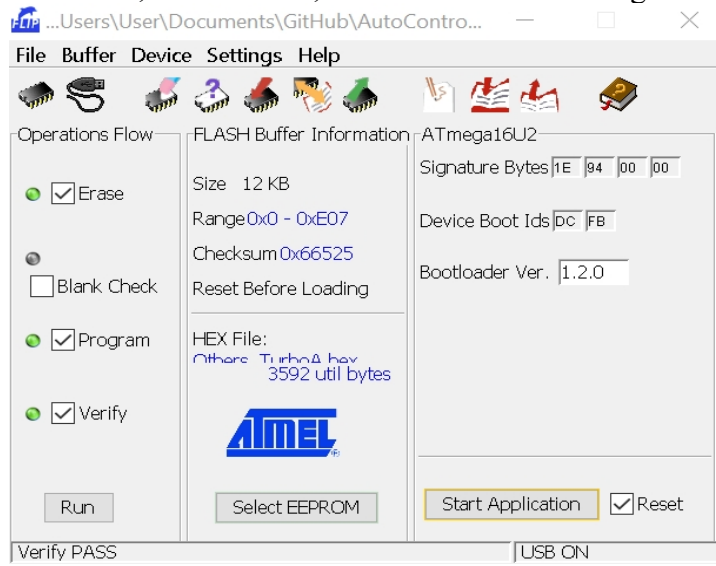


5. Press the 2nd icon, choose USB and press Open, if the Arduino has correctly entered DFU mode, it should be able to connect successfully, on the board you should see the RX and TX LED lights up



6. Go to File -> Load HEX file... and select **AutoController2_atmega16u2.hex** file provided by Auto Controller 2 download, if your directory that contains the hex file has Chinese/Japanese/Korean, this may fail to load, move them to or install Auto Controller 2 to a different directory

- Press Run, if successful, it should look something like this:

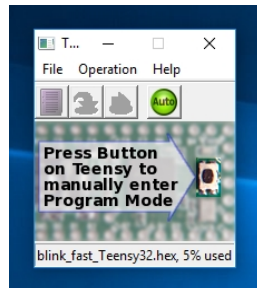


If a red light instead of green shows up next to Program, reopen FLIP and try again.

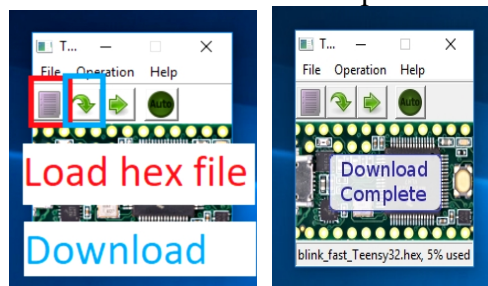
- Unplug the board from your computer

Teensy 2.0/Teensy++ 2.0:

- Download Teensy Loader from here: <https://www.pjrc.com/teensy/loader.html>
- Connect Teensy to your computer with USB 2.0 to USB mini adapter
- Press the button on Teensy to enter program mode, the first time you have to wait for Windows to install drivers



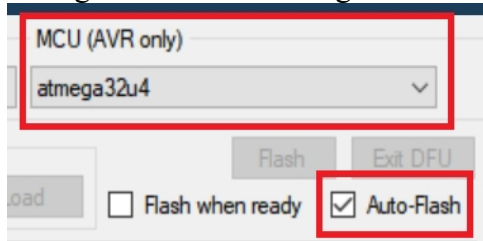
- Click to first icon to select **AutoController2_atmega32u4.hex** (Teensy 2.0) or **AutoController2_at90usb1286.hex** (Teensy++ 2.0) file provided by Auto Controller 2 download and press the 2nd icon to install the program into Teensy



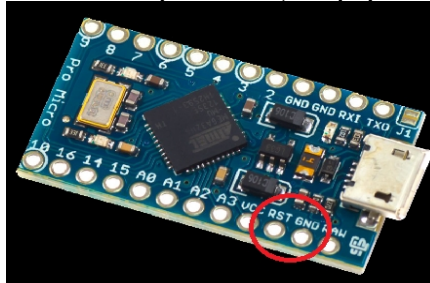
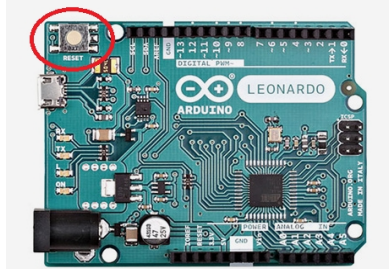
- Unplug the board from your computer

Arduino Pro Micro/Leonardo:

1. Download QMK Toolbox: https://github.com/qmk/qmk_toolbox/releases
2. Open the **AutoController2_atmega32u4.hex** file
3. Change the MCU to atmega32u4 and check the “Auto-Flash” checkbox



4. Connect Arduino Pro Micro/Leonardo to your computer with micro USB, you may need to wait for it to install driver the first time
5. Reset the board: For Leonardo, press the button on the top left; for Pro Micro, you need to briefly connect RST and GND pinholes (use paperclip or tweezers)



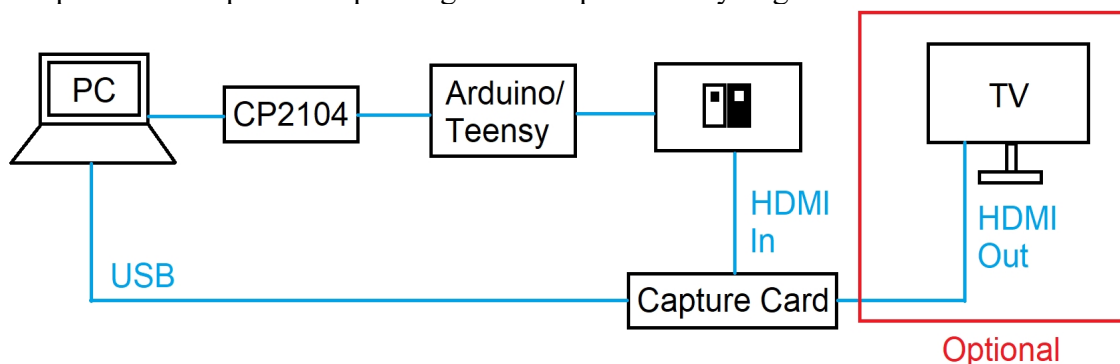
6. QMK Toolbox should automatically install the hex file into the board and should show a bunch of logging and should show this at the end:

```
avrdude.exe: verifying ...  
avrdude.exe: 8336 bytes of flash verified  
  
avrdude.exe done. Thank you.
```

7. Unplug the board from your computer

3.2 Connecting Everything Together

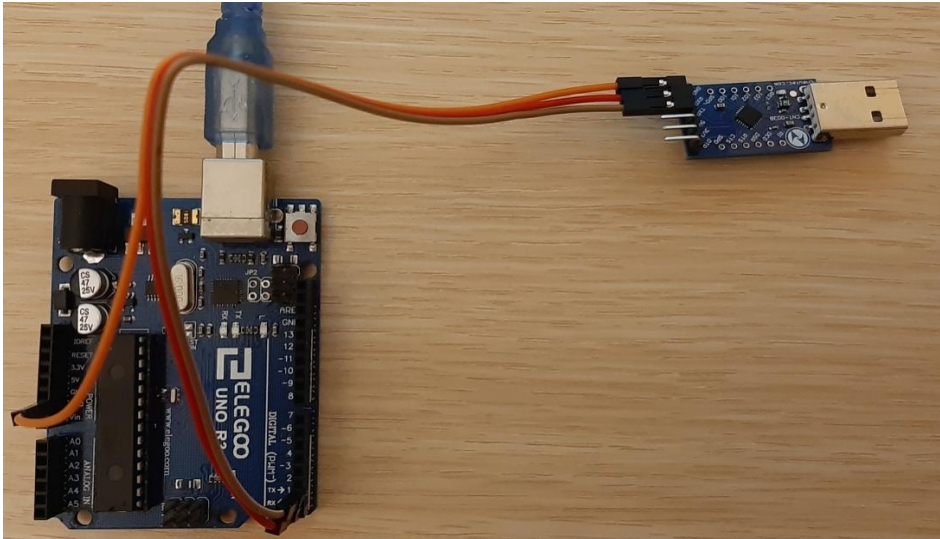
At the end of this section, you should expect to have connection like this diagram, some components are optional depending on the capture card you got.



Connecting CP2104 with Arduino Uno R3/Leonardo

Using the male-to-female jumper wires, make the following connections, **color of the wires do not matter as long as you are making the correct connections:**

- **TXD** pin on CP2104 to **TX -> 1** header on Arduino
- **RXD** pin on CP2104 to **RX <- 0** header on Arduino
- **GND** pin on CP2104 to any **GND** header on Arduino



Connecting CP2104 with Arduino Pro Micro

Using the female-to-female jumper wires and the mini-grabbers, make the following connections, **color of the wires do not matter as long as you are making the correct connections:**

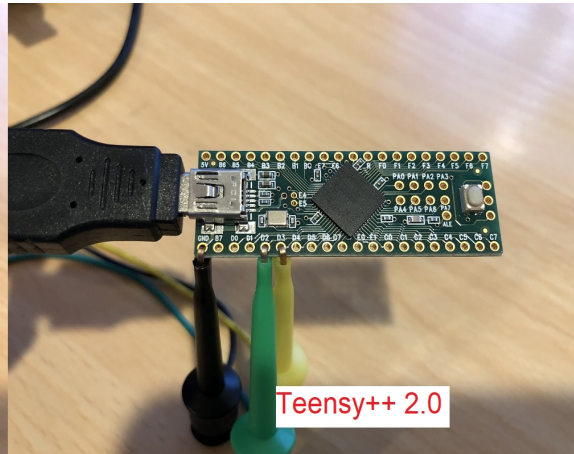
- **TXD** pin on CP2104 to **RX1** pinhole on Arduino
- **RXD** pin on CP2104 to **TX0** pinhole on Arduino
- **GND** pin on CP2104 to any **GND** pinhole on Arduino



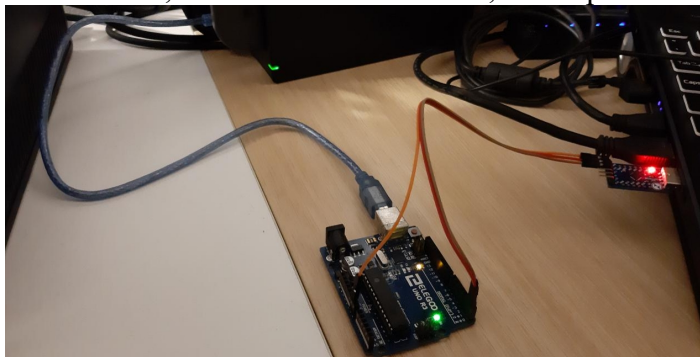
Connecting CP2104 with Teensy 2.0/Teensy++ 2.0

Using the female-to-female jumper wires and the mini-grabbers, make the following connections, **color of the wires do not matter as long as you are making the correct connections:**

- TXD pin on CP2104 to **D2** pinhole on Teensy
- RXD pin on CP2104 to **D3** pinhole on Teensy
- GND pin on CP2104 to any **GND** pinhole on Teensy



Once you have all the wiring finished, connect the USB end on Arduino/Teensy to the Switch dock, CP2104 USB end to PC, and capture card connected, you are ready to go.



4. Using Auto Controller 2

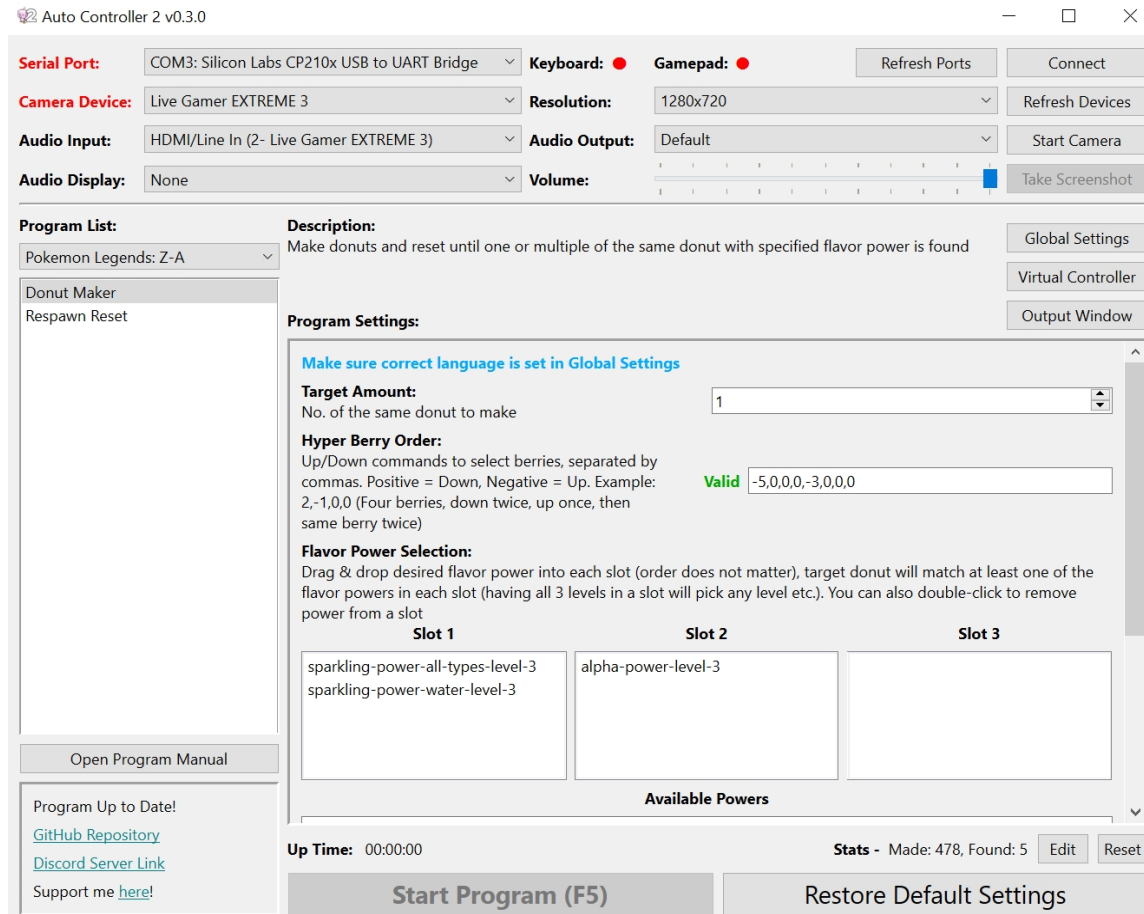
4.1 Disconnecting Other Controllers

Most games only allow one controller connected at a time, if you try to connect a 2nd one it will not connect. So you must disconnect the external controller you are playing with.

To disconnect, press the sync button on the wireless controller, note that if using detached joy-cons, you have to press the sync button on BOTH joy-cons. After unsyncing, the green light should be turned off. For wired controllers, simply unplug it from the Switch.



4.2 Basic Connection



Start by selecting the correct serial port, it should be for the CP210x chip, then press **Connect**, it will attempt to communicate with Arduino/Teensy and it will connect after 0.5s if feedback was established. If fails check the following:

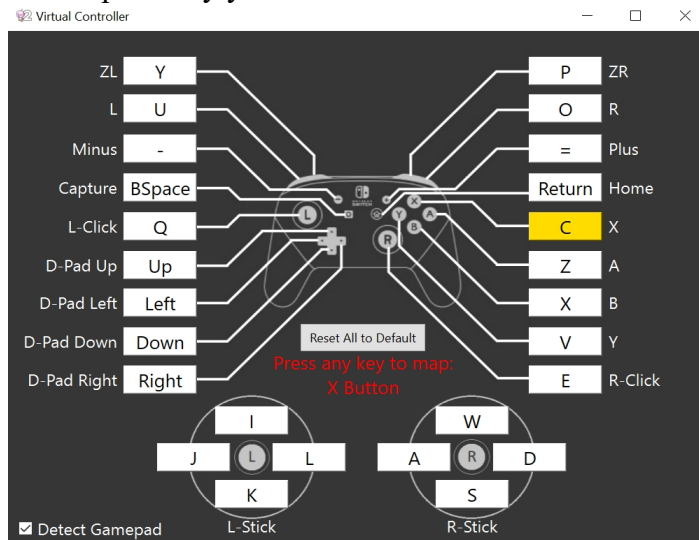
- Have .hex installed to the board
- Have correct wire connections between the board and CP210x chip
- Connect the board to the Switch AFTER disconnecting all other controllers

Next select the capture card device and press **Start Camera**, you must make sure no other applications are occupying the capture card (like OBS), otherwise it will not start. **If camera is showing weird color, it is likely Red/Blue channel getting swapped, go to Global Settings, under Development Settings, check Swap Red/Blue Channel.**

Each program may or may not require all components like Serial, Video and Audio. If the component is required it will show red text and you will be unable to start program.

4.2 Testing Keyboard Controls

With **Media View** or **Virtual Controller** window focused, now you should be able to control the game with keyboard and Gamepad (only tested for Xbox controller). Go to **Virtual Controller** and you can change the button layout by pressing on the key buttons and map the key you want.



If the control is completely wrong and doing weird stuff, this can happen if you got a 2nd hand Arduino Uno R3 board, there is a high chance the Atmega328p chip is interfering with the program. You will have to remove the chip in the picture below. Be careful to alternate prying on left and right to avoid breaking the pins.



If the chip is not detachable (SMD revision for Uno), you will need to do the following:

1. <https://support.arduino.cc/hc/en-us/articles/4408887452434-Flash-USB-to-serial-firmware-in-DFU-mode> Download the original Arduino firmware and flash it like you would with Flip
2. <https://docs.arduino.cc/learn/programming/sketches> Follow this guide and update an empty sketch to the board (with setup and loop empty)
3. Re-flash Smart Program hex file

4.3 Global Settings

Please go through the settings thoroughly, the most important is **System Settings** as different languages and Nintendo Switch system can run differently for each program.

System Settings

Language:
Language of the Nintendo Switch system or the current game. Required for OCR (Text Recognition)

English ▼

System:
Type of the current Nintendo Switch system. This can affect what command to be used

Nintendo Switch 2 ▼

Reset Default Settings

Go to section 5 for more details on **Discord Settings**

4.4 Nintendo Switch System Settings

There are a few settings on the Nintendo Switch system you must change.

1. Set Auto-Sleep (Connected to TV) in Sleep Mode section to Never
2. Under Display section, set RGB Range to Full Range
3. Under Display section, click Adjust Screen Size and set to 100%
4. For Nintendo Switch 2 users, you **MUST** turn off HDR output

4.5 Running a Program

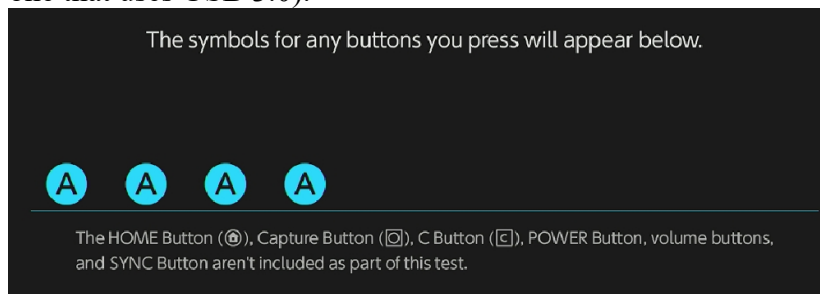
Select the program you want to run, press **Open Program Manual** (if it is enabled) and follow the instruction there, change the program settings accordingly.

Now you should be fully setup, press **Start Program** to start!

4.6 Testing Camera Delay

When you first launch Auto Controller 2, you are prompted to run **Camera Checker** program after reading this manual. This program ensures color is correct and camera delay is fast enough for programs to run properly. Start the program at Home screen.

It will first check your system type and theme color, and warn you if your Global Settings is set incorrectly or any issue with camera's color. Then it goes to **Test Controller Buttons** menu to check camera delay. If the camera delay is larger than 1 second, then some programs may not work properly. You may need a different capture card (preferably one that uses USB 3.0).



5. Discord Connection

5.1 Use Cases

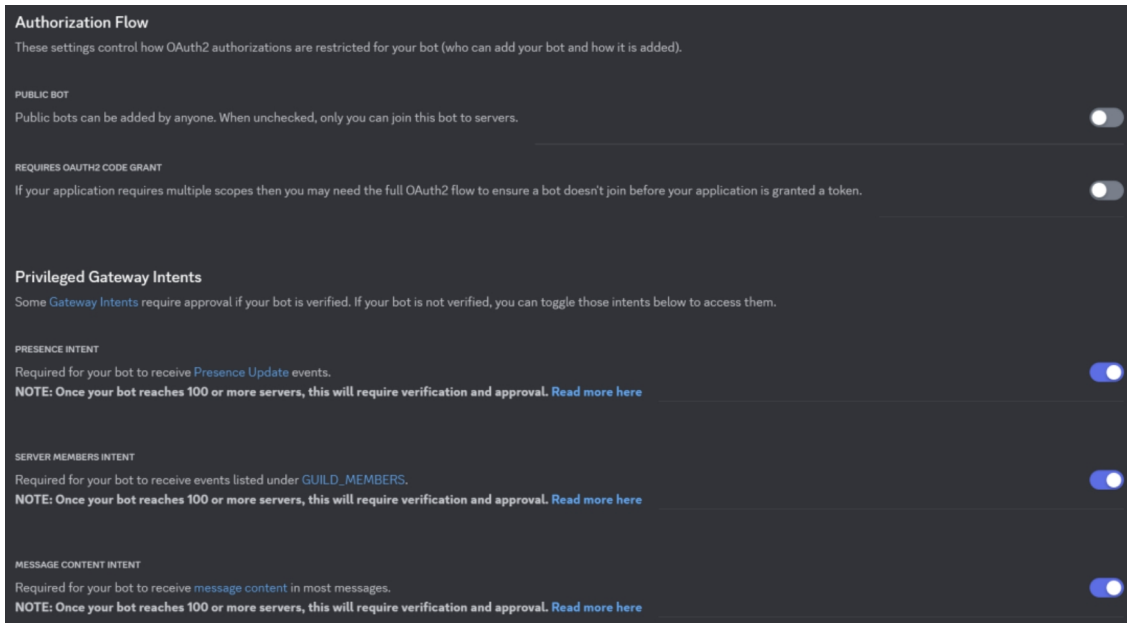
Discord bot connection which can be useful for:

- Get notification when a shiny is found
- Get notification every hour for long-running program
- Get notification when a program has finished successfully or failed
- Send remote controller input commands, take screenshots, current status etc.

Note: If you want to remote control Smart Program Manager it's best to use programs like Team Viewer to remote access your computer directly rather than using discord commands.

This will guide you how to setup your own Discord bot.

1. Go to Discord's Developer Portal and create a new application
2. Go to Bot tab (on the left), make sure to set your bot as private, and give the following Privileged Gateway Intents



The screenshot shows the 'Bot' tab settings in the Discord Developer Portal. It includes sections for 'Authorization Flow' and 'Privileged Gateway Intents'. The 'PUBLIC BOT' toggle is off. The 'REQUIRES OAUTH2 CODE GRANT' toggle is on. Under 'Privileged Gateway Intents', the 'PRESENCE INTENT', 'SERVER MEMBERS INTENT', and 'MESSAGE CONTENT INTENT' toggles are all turned on. Each intent section includes a note about requiring verification and approval once the bot reaches 100 servers.

Authorization Flow
These settings control how OAuth2 authorizations are restricted for your bot (who can add your bot and how it is added).

PUBLIC BOT
Public bots can be added by anyone. When unchecked, only you can join this bot to servers. ☐

REQUIRES OAUTH2 CODE GRANT
If your application requires multiple scopes then you may need the full OAuth2 flow to ensure a bot doesn't join before your application is granted a token. ☒

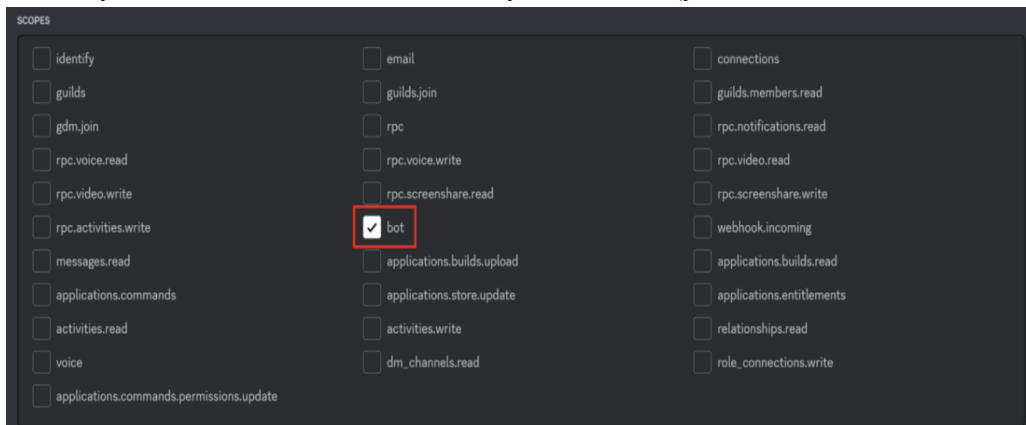
Privileged Gateway Intents
Some Gateway Intents require approval if your bot is verified. If your bot is not verified, you can toggle those intents below to access them.

PRESENCE INTENT
Required for your bot to receive Presence Update events. ☒
NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)

SERVER MEMBERS INTENT
Required for your bot to receive events listed under GUILD_MEMBERS. ☒
NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)

MESSAGE CONTENT INTENT
Required for your bot to receive message content in most messages. ☒
NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)

3. Go to OAuth2 tab and click URL Generator, you need this to generate an invite URL so you can invite the bot to one of your servers (you need to create one if not already)



The screenshot shows the 'Scopes' section in the Discord Developer Portal. It displays a list of permissions that can be granted to the bot. The 'bot' scope is selected and highlighted with a red box.

SCOPES

☐ identify	☐ email	☐ connections
☐ guilds	☐ guilds.join	☐ guilds.members.read
☐ gdm.join	☐ rpc	☐ rpc.notifications.read
☐ rpc.voice.read	☐ rpc.voice.write	☐ rpc.video.read
☐ rpc.video.write	☐ rpc.screenshare.read	☐ rpc.screenshare.write
☐ rpc.activities.write	☒ bot	☐ webhook.incoming
☐ messages.read	☐ applications.builds.upload	☐ applications.builds.read
☐ applications.commands	☐ applications.store.update	☐ applications.entitlements
☐ activities.read	☐ activities.write	☐ relationships.read
☐ voice	☐ dm_channels.read	☐ role_connections.write
☐ applications.commands.permissions.update		

BOT PERMISSIONS

GENERAL PERMISSIONS	TEXT PERMISSIONS	VOICE PERMISSIONS
☐ Administrator	☒ Send Messages	☐ Connect
☐ View Audit Log	☐ Create Public Threads	☐ Speak
☐ Manage Server	☐ Create Private Threads	☐ Video
☐ Manage Roles	☐ Send Messages in Threads	☐ Mute Members
☐ Manage Channels	☐ Send TTS Messages	☐ Deafen Members
☐ Kick Members	☐ Manage Messages	☐ Move Members
☐ Ban Members	☐ Manage Threads	☐ Use Voice Activity
☐ Create Instant Invite	☒ Embed Links	☐ Priority Speaker
☐ Change Nickname	☒ Attach Files	☐ Request To Speak
☐ Manage Nicknames	☒ Read Message History	☐ Use Embedded Activities
☐ Manage Expressions	☐ Mention Everyone	☐ Use Soundboard
☐ Create Expressions	☐ Use External Emojis	☐ Use External Sounds
☐ Manage Webhooks	☐ Use External Stickers	
☐ Read Messages/View Channels	☐ Add Reactions	
☐ Manage Events	☐ Use Slash Commands	
☐ Create Events	☐ Use Embedded Activities	
☐ Moderate Members		
☐ View Server Insights		
☐ View Creator Monetization Insights		

GENERATED URL

https://discord.com/api/oauth2/authorize?client_id=1167366&permissions=1167366&scope=bot Copy

4. Copy the URL, paste it to a browser and invite the Bot to your server

5.2 Discord Settings

Discord Settings

Bot Token:
Token of your discord bot, do not share this with anyone and keep it safe

User ID:
The bot will send program status to this user (shiny, finished, error etc.)

Channel ID:
The bot will only send special program status to this channel (shiny etc.), it must have appropriate permissions in that server

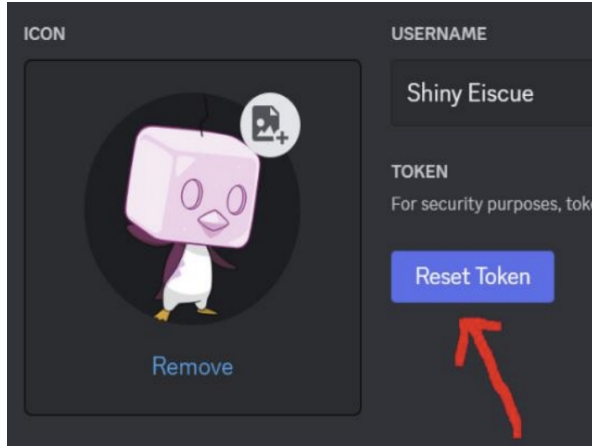
Toggle Bot:
Application will start bot automatically at launch if bot has started

Hourly Status Update:
Send Program Status message every hour to remind user a program is running

Finish Notification Suppression:
Prevent sending program finish/error notification if program duration is less than this many minutes

Reset Default Settings

- **Bot Token:** Go back to Bot tab, press **Reset Token** to reveal the token, copy it and save it somewhere safe, as it will not be shown until you reset the token again



- **User ID:** To obtain your ID you must enable **Developer Mode**, enable it in Settings->Advanced, then right-click on your icon and click **Copy User ID**, then paste the ID here
- **Channel ID:** This is the channel where special notification like shiny Pokemon will be sent to, right-click on one of the channels in your server and **Copy Channel ID**, then paste the ID here
- **Toggle Bot:** Enable/Disable bot, if enabled, next time Auto Controller 2 launch will automatically connects the bot
- **Hourly Status Update:** Send program status message every hour to remind user a program is running
- **Finish Notification Suppression:** Prevent sending program finish/error notification if program duration is less than this many minutes